

# The Token Optimization Hackathon — Submission

**Track:** B — Aarav's Workflow Diagnoser (no separate custom app; modified this repo)

**Submitted to:** code channel, Monday async check-in

---

## 1. The app + repo link

Modified Aarav diagnoser repo, retrieval layer implemented: <https://github.com/varunb1996/Tool-Calling>

- Naive (BEFORE) tools: `diagnoser/tools_naive.py` — `search_web` (`max_results=100`), new `search_gmail` + `search_calendar` tools dumping raw synthetic data
- RAG pipeline (AFTER): `diagnoser/rag/` — `chunk` → `index` → `keyword-filter` → `pre-aggregate`, wired behind the same three tool names
- Full writeup: `hackathon/REPORT.md`

Gmail/Calendar use synthetic data (2-year inbox, 248 emails + 190 events), not real OAuth — same reasoning the lecture used swapping in Open-Meteo for OpenWeatherMap: the pipeline is what's being tested, not auth plumbing.

## 2. BEFORE number

One representative query through the naive tools:

*"Search my email and calendar for anything related to the Q3 reporting automation project over the last year, and check the web for current no-code tools I could use — I need to scope my build brief."*

| Tool                                                           | Behavior                                         | Tokens        |
|----------------------------------------------------------------|--------------------------------------------------|---------------|
| <code>search_gmail</code> (naive)                              | ignores query, returns all 248 emails, full body | 24,986        |
| <code>search_calendar</code> (naive)                           | ignores query, returns all 190 events            | 12,661        |
| <code>search_web</code> (naive, <code>max_results=100</code> ) | 24 results, full untruncated content             | 7,727         |
| <b>Total tool output</b>                                       |                                                  | <b>45,374</b> |
| <b>Total payload (+ system prompt + user query)</b>            |                                                  | <b>45,964</b> |

Printed by tiktoken (cl100k\_base), reproducible: `python -m hackathon.measure_before` → [hackathon/before\\_results.json](hackathon/before_results.json)

## 3. AFTER number + reduction factor

Same query, same tool inputs, through the RAG pipeline:

| Tool                            | Behavior                                                                     | Tokens |
|---------------------------------|------------------------------------------------------------------------------|--------|
| <code>search_gmail</code> (RAG) | opened only <code>gmail_2025</code> , exact match: 28/28 real project emails | 1,959  |

|                          |                                                                   |              |
|--------------------------|-------------------------------------------------------------------|--------------|
| search_calendar (RAG)    | opened only calendar_2025, exact match: 10/10 real project events | 788          |
| search_web (RAG)         | top 5 of 100 by keyword relevance, 300-char snippets              | 585          |
| <b>Total tool output</b> |                                                                   | <b>3,332</b> |
| <b>Total payload</b>     |                                                                   | <b>3,922</b> |

**Reduction factor: 45,964 → 3,922 tokens = 11.7x compression** (13.6x on tool output alone). Zero false positives, zero missed matches on the synthetic data.

Reproducible: `python -m hackathon.measure_after` → [hackathon/after\\_results.json](#)

## 4. Index design

[diagnoser/rag/index.json](#) — one entry per year-chunk (6 total, 831 tokens for the whole index):

```
chunk_id, source, year, date_range {start, end}, record_count, labels, participants,
token_count, path
```

**Key added after seeing a real query:** the first retrieval pass matched on any single overlapping keyword and over-matched — 33 emails instead of 28, because an unrelated AI-newsletter email happened to contain the word "automation" on its own. There wasn't a missing index key so much as a missing matching rule: requiring 2+ overlapping keywords (instead of 1) once the query has 2+ terms fixed it to exact precision. Same lesson as the lecture's missing "month" key, found from the opposite direction — too loose a filter rather than too coarse a chunk.

## 5. Cost projection

Groq llama-3.3-70b-versatile: \$0.59 / million tokens.

|           | BEFORE (45,964 tok) | AFTER (3,922 tok) |
|-----------|---------------------|-------------------|
| Per query | \$0.02712           | \$0.00231         |

**Realistic usage** (occasional diagnosis sessions, ~1,500 queries total, one-time): \$40.68 → \$3.47, saves **\$37.21**. Does **not** clear the volume bar — too small to justify the engineering time on its own.

**If used daily** (500 active users × 1 query/day):

|               | BEFORE            | AFTER           | Savings           |
|---------------|-------------------|-----------------|-------------------|
| Daily         | \$13.56           | \$1.16          | \$12.40           |
| Monthly       | \$406.78          | \$34.71         | \$372.07          |
| <b>Yearly</b> | <b>\$4,949.17</b> | <b>\$422.30</b> | <b>\$4,526.87</b> |

**Verdict: clears the bar at daily-use volume.** And the case is stronger than the dollar figure alone shows: the synthetic inbox here is only 248 emails over 2 years — a real inbox is easily 10-100x that. Past a certain size the naive tool doesn't just cost more, it hits the same hard context-overflow wall the lecture's weather API did.

Full detail: [hackathon/REPORT.md](#)